TEC | Tecnológico de Costa Rica

Especificación del ISA - CERBERO

Escuela de Ingeniería en Computadores

CE 4301 - Arquitectura de computadores I

Profesor:
González Gomez Jefferson

Estudiantes:
Araya Porras Luis Diego - 2021086423
Braulio Retana Murillo - 2021050678
Michael Valverde Navarro - 2020044189
Piedra Montero Joseph - 2023047830

Grupo #4

Fecha de entrega:
Viernes 25 de septiembre del 2026

Semestre II, 2026

Nombre de la arquitectura: CERBERO.

Componente	CERBERO
Clase de ISA	Registro - Registro
Paradigma	VLIW, paralelismo explícito y calendarización estática
Encodificación	Variable a nivel de slot (16 y 32 bits), fija a nivel de bundle (128 bits)
Registros	16 ($R0 - R15$) de 32 bits cada uno
Modo de direccionamiento	6 (registro, inmediato, base+desplazamiento, base con postincremento, relativo al PC e indirecto por registro)
Licenciamiento	Abierto
Punto Flotante	No soportado

Justificaciones generales:

Clase ISA:

Se eligió registro-registro porque en una máquina de calendarización estática el generador de código necesita conocer la latencia de cada operación antes de ejecutarla. Si una operación de ALU pudiera leer memoria, su latencia depende del operando y no existiría un número único que respetar. Además, con cinco slots ejecutándose en paralelo, permitir accesos a memoria desde cualquier slot exigiría múltiples puertos a la memoria de datos y arbitraje entre ellos; un solo slot LSU significa un solo puerto.

Encodificación:

Las instrucciones de cifrado y de salto no requieren tantos bits como las aritmeticas o las de memoria, porque toman sus operandos de índices pequeños y de registros ya identificados.

Dimensionar cada slot según el contenido de información real de su unidad funcional reduce el bundle de 160 a 128 bits, un ahorro del 20% en memoria de instrucciones, y ataca la baja densidad de código que es la debilidad clásica de las arquitecturas VLIW.

Registros:

Ocho registros, el mínimo del enunciado, resultan ineficientes: con cinco slots emitiendo por ciclo y sin forwarding, los valores permanecen vivos más tiempo y el calendarizador se vería forzado a derramar memoria constantemente, ocupando el único slot LSU: Treinta y dos registros exigiría un campo de 5 bits, lo que rompería el slot criptográfico de 16 bits. Dieciséis es el punto donde el campo del registro cabe en 4 bits y queda margen de maniobra.

Modo de direccionamiento:

Se incluyen los modos registro e inmediato para que las operaciones trabajen directamente sobre el banco de registros, sin accesos adicionales a memoria. Para memoria se ofrecen bese+desplazamiento y con base con postincremento, ambos resueltos con una sola suma de un ciclo dentro de la LSU, lo que evita etapas de cálculo de dirección efectiva y mantiene una latencia de acceso determinista. Para control de flujo se usan el relativo al PC y el indirecto por registro. Se descartó el indexado registro+registro porque con dos ALU disponibles el cálculo de índices ocupa slots que de todos modos estarían vacíos.

Licenciamiento:

Una ISA propia y abierta permite añadir instrucciones dedicadas, como las rondas de Feistel4 y la gestión de la bóveda de llaves, sin depender de licencias. Además, nos garantiza el control total sobre la herramienta de ensamblado y nos facilita el contrato de interfaz con el grupo de compiladores.

Endianness:

Se decidió utilizar Little-endian, que consiste en guardar el byte de menos significativo en la primera dirección de memoria. Esto se considera más eficiente para el hardware, debido a que los cálculos de suma que implican hacer acarreo, se pueden hacer de manera directa en el hardware y se procesan de forma más limpia.

1. Modelo de propagación:

a. Banco de registros y convención de llamadas:

16 registros de propósito general de 32 bits. EL ancho del campo de registro es de 4 bits. Ninguno es cableado a un valor constante.

Registro	**Código**	**Uso**	**Reservado por el llamado**
$R0$, $R1$	0000, 0001	Par $P0$: bloque criptográfico A	No
$R2$, $R3$	0010, 0011	Par $P1$: bloque criptográfico B	No
$R4$, $R5$	0100, 0101	Par $P3$: bloque criptográfico C	No
$R6 - R9$	$0110 - 1001$	Argumentos 1 a 4. $R6$ lleva además el valor de retorno	No
$R10$, $R11$	1010, 1011	Temporales	No
$R12$	1100	Puntero de lectura	Sí
$R13$	1101	Puntero de escritura	Sí
$R14$	1110	Stack pointer	Sí
$R15$	1111	Registro de enlace, escrito por JAL y $JALR$	Sí

Solo $R15$ tiene significado arquitectónico. El resto de la tabla es convención de software acordada con el grupo CE 1108, y el hardware no la impone.

Pares criptográficos. Las rondas de Feistel 4 operan sobre pares alineados $P0$ - $P7$, donde $Pn = R(2n) : R(2n + 1)$, con la mitad L en el registro par y R en el impar. $P3$ a $P7$ no deben usarse como pares criptográficos: $P7$ es $R14 : R15$, y una ronda sobre ese par destruiría el stack pointer y la dirección de retorno.
Pila: Crece hacia direcciones menores. $R14$ apunta el último elemento ocupado. Los argumentos más allá del cuarto se pasan por pila, en orden inverso.

Protocolo de llamada: El llamador coloca los argumentos en $R6 - R9$ y ejecuta $jal. al$. Una función que realice llamadas anidadas debe guardar $R15$ al entrar ($subi\ r14,\ r14,\ \#4$ y luego $sw\ r15,\ \#0(r14)$) y restaurarlo al salir. Una función hoja retorna directamente con $jr. al\ r15$.

b. Contador de programa y registro de estado:

PC de 32 bits, siempre múltiplo de 16, dado que cada bundle ocupa 16 bytes. El avance secuencial es $PC \leftarrow PC + 16$.

Bits	Campo	Descripción	Escrito por
0	Z	Resultado cero	CMP, $CMPI$, $TEST$, $TESTI$ en $S0$
1	N	Resultado negativo	CMP, $CMPI$ en $S0$
2	C	Acarreo o préstamo	CMP, $CMPI$ en $S0$
3	V	Desbordamiento con signo	CMP, $CMPI$ en $S0$
4	$AUTH$	Sesión de bóveda abierta	Controlador de la bóveda

5	$AUTHFAIL$	El último $LOGIN$ fallo	Controlador de la bóveda
6	EXC	Ocurrio una falla	Cualquier unidad
$9:7$	$CAUSE$	Código de causa	Cualquier unidad
$31:10$	–	Reservado, se lee como cero	–

El PSW se lee con $MFPSW\ rd$ y no es escribible directamente.

Política de fallas: No se implementan excepciones con vector ni retorno de excepción. Toda condición anómala anula la operación del slot afectado y la registra en el PSW; el pipeline nunca se detiene ni se vacía.

$CAUSE$	**Nombre**	**Condición**	**Efecto**
000	–	Sin falla	–
001	$ILLOP$	Opcode no definido, campo reservado distinto de cero o comparación en S1	El slot actua como NOP
010	$MISALIGN$	Acceso a memoria desalineado	Se anula el acceso
011	$RANGE$	Dirección fuera del rango implementado	Se anula el acceso
100	$DENIED$	Operación de bóveda con $AUTH = 0$	Se anula la operación
101	$NOKEY$	Ronda Feistel4 sobre ranura con $KVALID = 0$	Se anula la ronda

110	$WCONF$	Dos slots escriben el mismo registro	Prevalece el slot de menor índice: $S0 > S1 > S2 > S3 > S4$
111	–	Reservado	–

Si dos slots fallan en el mismo bundle prevalece la causa del slot de índice menor y EXC se marca solo una vex. Cualquier falla limpia $AUTH$ de inmediato. Un $LOGIN$ fallido no es falla: marca $AUTHFAIL$ y se consulta con $BR.AUTH$ o con $MFPSW$.

Se eligió esta política sobre el mecanismo de trap porque este último exigiría vector, registro EPC, instrucción de retorno, vaciado de las etapas en vuelo y una regla para el caso de dos slots fallando a la vez. Anular y marcar cumple lo que pide el enunciado sin introducir ninguna ruta de control capaz de detener el pipeline.

c. Tipos de datos:

Tipo	**Ancho**	**Soporte**
Byte, con signo y sin signo	8 b	LH, LBU, SB
Media palabra, con y sin signo	16 b	LH, LHU, SH
Palabra	32 b	Todas las operaciones de ALU, LW, SW
Bloque criptográfico	64 b	Par de registro alineado, $F4E$, $F4D$
Llave	128 b	Bóveda, instalada en cuatro operaciones de 32 bits

Las variantes con y sin signo de los accesos cortos no son redundantes: un byte de un archivo es un valor sin signo de 0 a 255, mientras que un byte de un arreglo de enteros

pequeños tiene signo. Sin LBU, leer el byte $0xFF$ de un archivo daría -1 y el cifrado produciría resultados incorrectos.

No se soporta el punto flotante. El dominio de aplicación es cifrado por bloques y manipulación de archivos: operaciones enteras, lógicas y re rotación sobre la palabra de 32 bits. Incluirlo exigiría un espacio de opcodes adicional, un banco de registros propio o una convención para compartir el existente, y una unidad de latencia múltiple que rompería la regla de latencia uniforme que constituye el contrato con CE 1108.

d. Organización de memoria:

Arquitectura Harvard: memorias de instrucciones y de datos separadas con espacios de direcciones independientes.

Memoria	**Ancho de palabra**	**Direccionamiento**	**Tamaño**
$IMEM$	128 bits (un bundle)	Por bundle, PC múltiplo de 16	32 KB = 2048 bundles
DEM	8 bits	Por byte, direcciones de 32 bits	64 KB, parametrizable

Alineación natural obligatoria; un acceso desalineado se anula y registra $MISALIGN$. los rangos no implementados registran $RANGE$.

Se eligió Harvard porque el ancho del bundle no coincide con el de los datos: una memoria unificada exigiría un puerto de 128 bits y lógica de alineación para el acceso a datos. Adempas permite que la herramienta de carga de archivos escriba sobre $DMEM$ sin riesgo de corromper el programa.

Mapa de memoria de datos:

Rango	Tamaño	Contenido

$0x0000_0000 - 0x000_3FFF$	16 KB	Variables globales. Segmento que asigna el compilador de CE 1108
$0x0000_4000 - 0x000_BFFF$	32 KB	Buffers de datos: archivos a cifrar y descifrar
$0x0000_C000 - 0x000_EFFF$	12 KB	Reservado para heap; sin uso en la primera versión
$0x000_F000 - 0x000_FFFF$	4 KB	Pila; crece hacia abajo desde $0x000_FFFC$

Mapa de memoria de instrucciones

Rango	Tamaño	Contenido
$0x0000_0000 - 0x000_6FFF$	28 KB	Código de usuario
$0x0000_7000 - 0x000_7FFF$	4 KB	Región segura: SEC_BASE a SEC_LIMITE

La región segura es el único rango donde $AUTH$ puede permanecer encendido.

2. Encodificación:

a. Tipos de instrucción:

Tipo	**Descripción**
A	Aritmetico-lógico de registro
I	Aritmetico-lógico con inmediato

M	Acceso a memoria
F	Rondas de Feistel
V	Bóveda y seguridad
AC	ALU corta
C	Salto relativo
J	Salto indirecto

b. Campos comunes

Campo	**Ancho**	**Qué contiene**
opcode	5 bits (operaciones de 32 bits) / 3 bits (operaciones de 16 bits)	Identifica la operación dentro del espacio de opcodes de ese slot. Cada slot tiene su propio espacio, así que el mismo valor significa cosas distintas según la posición
rd	4 bits	Registro destino: dónde se escribe el resultado. $0000 = R0$, $1111 = R15$
rs1, rs2, rs	4 bits	Registros fuente: de donde se leen los operandos
reservado	variable	Bits sin uso en este formato. Deben codificarse en cero; un valor distinto registra $ILLOP$

c. Formato del bundle:

127	96	95	64	63	32	31	16	15	0
S0: ALU-0 (32)		S1: ALU-1 (32)		S2: LSU (32)		S3: CRP (16)		S4: BRU (16)	

Slots	Bits	Unidad funcional	Ancho
S0	$127:96$	ALU-0, única que escribe banderas	32 b
S1	$95:64$	ALU-1	32 b
S2	$63:32$	LSU	32 b
S3	$31:16$	Criptográfica, bóveda y ALU corta	16 b
S45	$15:0$	BRU	16 b

Los cinco slots se decodifican y despachan en el mismo ciclo y se ejecutan en paralelo. La asignación es fija: cada slot está atado permanentemente a su unidad funcional y no existe campo de selección.

El opcode 0 es NOP en los cinco espacios, así que un bundle de 128 ceros es un bundle vacío válido. $S0$ ocupa los bits más significativos, de modo que un volcado hexadecimal se lee en el mismo orden en que se escriben los slots en el ensamblador.

d. Formato de 32 bits:

	31	27	26	23	22	19	18	15	14	0
Tipo A	opcode		rd		rs1		rs2		reservado	
Tipo I	opcode		rd		rs1		imm19 (con signo)			

Tipo M	opcode	rd/rs_dato	rbase	imm19 (con signo)

- **Tipo A – Aritmético-lógico de registro:**

Campo	Descripción
opcode	Operación de ALU: ADD, XOR, SLL, CMP … Bit 4 en 0 indica forma de registro
rd	Registro destino
rs1	Primer operando
rs2	Segundo operando
reservado	15 bits sin usar. El formato requiere 17 de los 32 disponibles; el espacio queda para extensiones futuras

- **Tipo I – Aritmetico-logico con inmediato:**

Campo	Descripción
opcode	Misma operación que el tipo A con el bit 4 en 1: $ADDI$ es ADD con ese bit encendido
rd	Registro destino
rs1	Único operando de registro
imm19	Constante de 19 bits con extensión de signo, rango -262144 a 262143

En desplazamientos y rotaciones solo se usan los bits $[4:0]$ del inmediato, porque desplazar una palabra de 32 bits más de 31 posiciones no tiene sentido.

- **Tipo M – Acceso a memoria:**

Campo	Descripción
opcode	Tipo de acceso. Sus bits están estructurados: bit 4 = postincremento, bit 3 = store, bits $2{:}1$ = tamaño del dato, bit 0 = extensión con ceros
rd / rs_dato	En un load es el destino del dato leído; en un store es la fuente del dato a escribir. Es el mismo campo, el bit 3 del opcode decide cómo interpretarlo
rbase	Registro que contiene la dirección base. En las variantes $.INC$ también se escribe de vuelta con el valor incrementado
imm19	Desplazamiento en bytes, con signo, que suma a la base

La disposición de bits es idéntica al tipo I, pero se documenta aparte porque la semántica y el decodificador son distintos.

e. Formato de 16 bits:

	15	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Tipo F	opcode		par			kv		ronda		reservado					
Tipo V	opcode		kv / widx / subfn		widx/kv		rs			reservado					
Tipo AC	opcode		i	subop			rd			rs2/imm5					
Tipo C	opcode		cond			desp (con signo, en bundles)									

Tipo J	opcode	cond	rsv	rs	reservado

- **Tipo F – Ronda Feistel4:**

Campo	Descripción
opcode	$001 = F4E$ (cifrado), $010 = F4D$ (descifrado)
par	Par de registros alineado $P0 - P7$ que contiene el bloque de 64 bits. $Pn = R(2n) : R(2n + 1)$, con la mitad L en el registro par y R en el impar. Tres bits en lugar de dos campos de registro de 4
kv	Cuál de las 4 llaves de la bóveda usar (0 a 3)
ronda	Cuál de las 4 subllaves de esa llave aplicar (0 a 3), es decir, el número de ronda
reservado	6 bits en cero

No hay campo de registro porque par identifica a los dos a la vez, y la subllave nunca viene de un registro: sale de la bóveda, que es lo que exige el enunciado.

- **Tipo V – Bóveda y seguridad:**

Campo	Descripción
opcode	$011 = KSETW$, $100 = AUTHW$, $101 = VCTL$
kv	Índice de llave dentro de la bóveda (0 a 3). En $KSETW$ ocupa $[12: 11]$; en $VCTL$ ocupa $[10: 9]$

widx	Índice de palabra dentro de la llave o de la credencial (0 a 3). Una llave de 128 bits se instala en cuatro operaciones de 32 bits
subfn	Solo en $VCTL$: $00 = LOGIN$, $01 = LOGOUT$, $10 = PWSET$, $11 = KCLR$
rs	Registro que aporta la palabra de 32 bits, sea de llave ($KSETW$) o de credencial ($AUTHW$). Sin usar en $VCTL$
reservado	5 bits en cero

Los campos $[12:11]$ y $[10:9]$ cambian de significado según el opcode. Es la única parte del ISA donde ocurre, y se hace porque ninguna operación de bóveda necesita kv, $widx$ y $subfn$ al mismo tiempo.

- **Tipo AC – ALU corta:**

Campo	**Descripción**
opcode	Siempre 110: identifica a la ALU corta dentro del slot S3
i	Selecciona la forma del segundo operando: 0 = registro, 1 = inmediato
subop	Operación concreta: $ADD.S$, $XOR.S$, $MOV.S$ … Ocho por cada valor de i
rd	Registro que es fuente y destino a la vez. El formato es destructivo: $rd \leftarrow rd\ op\ operando$
rs2 / imm5	Con $i = 0$, registro fuente en los bits $[3:0]$ y el bit $[4]$ reservado. Con $i = 1$, inmediato sin signo de 5 bits (0 a 31)

- Es destructivo porque en los 16 bits no caben tres campos de registro. Ese es el precio de meter una ALU en un slot corto, y es lo que permite que los programas sin cifrado no desperdicien el slot S3.
- No escribe banderas: las banderas son exclusivas de S0.

- **Tipo C – Salto relativo:**

Campo	Descripción
opcode	$001 = BR$, $010 = JAL$
cond	Condición evaluada sobre el PSW: AL, EQ, NE, LT, GE, LTU, GEU, $AUTH$. Con AL es salto incondicional
desp	Desplazamiento relativo al PC, con signo, contado en bundles y no en bytes. Rango $\pm$ 512 bundles, equivalente a $\pm$ 8 KB

- Destino: $PC + 16 \times (1 + signext(desp))$. Se cuenta en bundles porque todo bundle mide 16 bytes, así que los cuatro bits bajo de un desplazamiento en bytes siempre serían cero: codificarlos sería desperdiciar alcance.
- No hay campos de registro porque la condición ya está calculada en PSW por una instrucción de comparación previa. Eso es lo que permite que el slot quepa en 16 bits.

- **Tipo J – Salto indirecto:**

Campo	Descripción
opcode	$011 = JR$, $100 = JALR$, $101 = HALT$
cond	Misma tabla de condiciones que el tipo C
rs	Registro que contiene la dirección destino. En un retorno de función es $R15$
reservado	Bit $[9]$ y bits $[4:0]$ en cero. $HALT$ no usa

	ningún campo salvo el opcode

Existe porque el tipo C solo alcanza $\pm\ 8KB$. Los saltos largos, el retorno de función y las llamadas por puntero necesitan una dirección completa de 32 bits, que solo cabe en un registro.

f. Reglas transversales:

- Posición: En todos los formatos cortos, el campo de registro principal ocupa siempre los bits $[8{:}\,5]$. El decodificador de S3 y S4 extrae ese rango sin lógica condicional.
- Reservados: Todo bit marcado como reservado debe codificarse en cero. Un valor distinto de cero anula la operación del slot y registra $ILLOP$.

3. Slot ALU (S0, S1):

El bit 4 del opcode selecciona la forma del segundo operando y los bits $3{:}\,0$ la operación, de modo que el decodificador en un $case$ de más de 4 bits más un multiplexor de un bit. Cuatro casillas no siguen la simetría y se documentan como excepciones: 10000 (NEG), 10011 y 11011 (reservados) y 11101 $(MOVH)$.

Mnemónico	Sintaxis	Descripción	Tipo	Opcode	Posición en bundle
NOP	nop	Operación nula (sin efecto en los registros)	A	00000	S0,S1
ADD	add rd, rs1, rs2	Suma aritmética	A	00001	S0,S1
SUB	sub rd, rs1, rs2	Resta aritmética	A	00010	S0,S1
MUL	mul rd, rs1, rs2	Multiplicación, 32 bits bajos	A	00011	S0,S1
AND	and rd, rs1, rs2	AND lógico	A	00100	S0,S1

OR	or rd, rs1, rs2	OR lógico	A	00101	S0,S1
XOR	xor rd, rs1, rs2	XOR lógico	A	00110	S0,S1
SLL	sll rd, rs1, rs2	Desplazamiento lógico a la izquierda	A	00111	S0,S1
SRL	srl rd, rs1, rs2	Desplazamiento lógico a la derecha	A	01000	S0,S1
SRA	sra rd, rs1, rs2	Desplazamiento aritmético a la derecha	A	01001	S0,S1
ROL	rol rd, rs1, rs2	Rotación a la izquierda	A	01010	S0,S1
ROR	ror rd, rs1, rs2	Rotación a la derecha	A	01011	S0,S1
MOV	mov rd, rs1	$R[rd] \leftarrow R[rs1]$	A	01100	S0,S1
NOT	not rd, rs1	$R[rd] \leftarrow \sim R[rs1]$	A	01101	S0,S1
CMP	cmp rs1, rs2	$R[rs1] - R[rs2]$, resultado descartado	A	01110	Solo S0
TEST	test rs1, rs2	$R[rs1]$ & $R[rs2]$, resultado descartado	A	01111	Solo S0
NEG	neg rd, rs1	$R[rd] \leftarrow - R[rs1]$	A	10000	S0,S1
ADDI	addi rd, rs1, #i	Suma con inmediato	I	10001	S0,S1
SUBI	subi rd, rs1, #i	Resta con inmediato	I	10010	S0,S1
–		Reservado		10011	
ANDI	andi rd, rs1, #i	AND con un inmediato	I	10100	S0,S1
ORI	ori rd, rs1, #i	OR con un inmediato	I	10101	S0,S1
XORI	xori rd, rs1, #i	XOR con un inmediato	I	10110	S0,S1
SLLI	slli rd, rs1, #i	Desplazamiento lógico a la izquierda con inmediato	I	10111	S0,S1

SRLI	srli rd, rs1, #i	Desplazamiento lógico a la derecha con inmediato	I	11000	S0,S1
SRAI	srai rd, rs1, #i	Desplazamiento Aritmético a la Derecha con un Inmediato	I	11001	S0,S1
ROLI	roli rd, rs, #i	Rotación izquierda inmediata	I	11010	S0,S1
–		Reservado		11011	
MOVI	movi rd, #i	Carga inmediato con signo en registro	I	11100	S0,S1
MOVH	movh rd, #i	Cargar un número constante (un inmediato) directamente dentro de un registro	I	11101	S0,S1
CMPI	cmpi rs1, #i	Comparar directamente el contenido de un registro con un número constante	I	11110	Solo S0
TESTI	testi rs1, #i	Prueba bits contra un inmediato, resultado descartado	I	11111	Solo S0

- No existe RORI: el ensamblador lo sintetiza como roli rd, rs1, #(32-k).
- Para construir una constante de 32 bits el orden obligatorio es MOVI y luego MOVH.

4. Slot LSU (S2):

Estructura del opcode:

opcode[4] = INC	1 = escribe de vuelta en el registro base
opcode[3] = ST	0 = load, 1 = store

opcode[2:1] = TAM	01 = byte, 10 = media palabra, 11 = palabra
opcode[0] = U	1 = extensión con cero (solo loads)

Mnemónico	Sintaxis	Descripción	Opcode
NOP	nop	Operación nula	00000
LB	lb rd, #i(rbase)	Carga de un byte a un registro de propósito general (es signed, importa el signo del byte que se carga)	00010
LBU	lbu rd, #i(rbase)	Carga de un byte a un registro de propósito general (es unsigned, NO importa el signo del byte que se carga)	00011
LH	lh rd, #i(rbase)	Carga media palabra con signo	00100
LHU	lhu rd, #i(rbase)	Carga media palabra sin signo	00101
LW	lw rd, #i(rbase)	Carga de un word (32 bits) a un registro de propósito general	00110
SB	sb rd, #i(rbase)	Operación de almacenamiento de un byte de un registro a memoria	01010

		RAM	
SH	sh rd #i(rbase)	Almacena media palabra	01100
SW	sw rd #i(rbase)	Operación de almacenamiento de un word de un registro a memoria RAM	01110
LW.INC	lw.inc rd, #i(rbase)	Carga palabra con postincremento	10110
SW.INC	sw.inc rs_dato, #i(rbase)	Almacena palabra con postincremento	11110

Dirección efectiva. En las instrucciones sin postincremento, $EA = R[rbase] + signext(imm19)$. En las variantes $.INC$ la semántica es diferente y debe respetarse:

Instrucción	**Dirección efectiva**	**Efectos sobre** $rbase$
$LW\ rd,\ \#i(rbase)$	$R[rbase] + signext(i)$	Ninguno
$LW.INC\ rd,\ \#i(rbase)$	$R[rbase]$, sin desplazamiento	$R[rbase] \leftarrow R[rbase] + signext(i)$

- Es decir, las variantes $.INC$ son post-incremento: acceden a la dirección actual y luego avanzan el puntero. El inmediato es el incremento, no un desplazamiento de acceso.
- Caso límite: Si $rd == base$ en un $LW.INC$, prevalece el dato cargado y el incremento se descarta.
- El postincremento permite que el acceso a memoria y la actualización del puntero ocurran en el mismo ciclo dentro de $S2$, liberando las dos ALU. Su costo es un segundo puntero de escritura en la LSU. Los opcodes con $TAM = 00$ y las combinaciones INC no listadas quedan reservados.

5. Slot criptográfico (S3):

Instrucción	Tipo	Opcode
NOP	–	000
F4E	F	001
F4D	F	010
KSETW	V	011
AUTHW	V	100
VCTL	V	101
ALU corta	AC	110
Reservado	–	111

a. Tipo F – rondas Feistel4:

Mnemónico	Sintaxis	Descripción	Opcode	$AUTH$
F4E	f4e par, kv, #ronda	Ejecuta una ronda de cifrado sobre el par indicado usando la subllave $ronda$ de la llave kv que se almacena en la bóveda. El nuevo par (L, R) se reescribe en	001	Sí

		los mismos registros		
F4D	f4d par, kv, #ronda	Ejecuta una ronda de descifrado sobre par, usando la subllave ronda de la llave kv. Es igual que F4E pero se aplica la función inversa	010	Sí

b. Tipo V - Bóveda:

Mnemónico	**Sintaxis**	**Descripción**	**Opcode**	$AUTH$
KSETW	ksetw kv, #widx, rs	Escribe la palabra $widx$ de la llave kv con el valor de rs. $KVALID[kv]$ se fija cuando las 4 palabras están escritas	011	Sí
AUTHW	authw #widx, rs	Escribe la palabra $widx$ del registro de desafío de 128 bits, que es solo de escritura	100	No

VCTL	según subfunción	Control de sesión y de ranuras	101	según subfn

Subfunciones de VCTL

Mnemónico	**Sintaxis**	**Descripción**	$subfn$	$AUTH$
$LOGIN$	login	Compara el registro de desafío con ROT_SECRET. Si coincide se pone AUTH = 1; si no, AUTHFAIL = 1	00	No
$LOGOUT$	logout	Se pone AUTH = 0, de forma manual y limpia el registro de desafío	01	No
$PWSET$	pwset	$ROT_SECRET \leftarrow registro\ de\ desafío$	10	Sí
$KCLR$	kclr kv	Borra $VAULT[kv]$ por completo y pone $KVALID[kv]$ en 0	11	Sí

$LOGIN$ y $PWSET$ no llevan operando de registro: Operan sobre el registro de desafío que $AUTHW$ llenó en las cuatro instrucciones previas. La credencial es de 128 bits, igual que las llaves.

c. Tipo AC – ALU corta:

Formato destructivo $rd \leftarrow rd\ op\ operando$. No escribe banderas

subop	$i = 0$ **(registro)**	$i = 1$ **(inmediato de 5 bits, sin signo)**
000	add.s rd, rs	addi.s rd, #imm5
001	sub.s rd, rs	subi.s rd, #imm5
010	and.s rd, rs	andi.s rd, #imm5
011	or.s rd, rs	ori.s rd, #imm5
100	xor.s rd, rs	xori.s rd, #imm5
101	mov.s rd, rs	movi.s rd, #imm5
110	not.s rd, rs	Reservado
111	neg.s rd, rs	Reservado

- No se incluyen los desplazamientos ni la multiplicación en la ALU corta: un desplazador de barril y un multiplicador exceden el presupuesto de área de un slot secundario. Las operaciones que quedan reutilizan el sumador y las compuertas lógicas que la unidad ya no necesita.
- La ALU corta existe para que los programas que no usan la unidad criptográfica no desperdicien un quinto de cada bundle emitiendo solo NOP en S3.

6. Slot BRU (S4):

Mnemónico	Sintaxis	Descripción	Tipo	Opcode
NOP	nop	Operación nula	–	000
BR	br.cond #desp	Si se cumple cond, $PC \leftarrow PC + 16 \times (1 + signext(desp))$	C	001
JAL	jal.cond #desp	$R15 \leftarrow PC + 32$; luego salta como BR	C	010
JR	jr.cond rs	Si se cumple cond, $PC \leftarrow R[rs]$	J	011
JALR	jalr.cond rs	$R15 \leftarrow PC + 32$; luego salta como JR	J	100
HALT	halt	Detiene la ejecución y finaliza la simulación	J	101
Reservado			–	110-111

El enlace es $PC + 32$ y no $PC + 16$ porque el bundle siguiente es el delay slot y ya se ejecutó: el retorno debe apuntar al bundle posterior a él.

Código de condición:

cond	Mnemónico	Condición sobre el PSW
000	AL	Siempre
001	EQ	$Z = 1$
010	NE	$Z = 0$
011	LT	$N \neq V$
100	GE	$N = V$

101	LTU	$C = 1$
110	GEU	$C = 0$
111	AUTH	$AUTH = 1$

NO existen BEQ, BNE, BLT ni JMP como instrucciones separadas. Se construyen así:

Forma equivalente	Secuencia en la arquitectura
beq rs1, rs2, L	cmp rs1, rs2 en S0, un bundle de separación, br.eq L
bne rs1, rs2, L	cmp rs1, rs2 en S0, un bundle de separación, br.ne L
blt rs1, rs2, L	cmp rs1, rs2 en S0, un bundle de separación, br.lt L
jmp L	br.al L
Retorno de función	jr.al r15

Se detalla de mejor manera en la sección 11.

Resumen por Slot:

Slot	Tipos	Rango de opcode	Mnemónicos
S0 (ALU-0)	A / I	$00000 - 11111$	Los 30 de la sección 3
S1 (ALU-1)	A / I	$00000 - 11111$	Los mismo excepto $CMP, TEST, CMPI$ y $TESTI$

S2 (LSU)	M	$00000 - 11110$	
S3 (Cripto / Bóveda / ALU corta)	F / V / AC	$000 - 110$	
S4 (BRU)	C / J	$000 - 101$	

7. Aplicación criptográfica - Feistel4:

La red de Feistel opera sobre bloques de 64 bits divididos en dos mitades L y R de 32 bits, aplicando cuatro rondas con una subllave distinta por ronda.

$ROL32(x,\ r) = ((x <<) \mid (x >> 32 - r)))$
$F(x,\ k) = (ROL(x,\ 5) + k)\ \hat{}\ ROL(x,\ 13)$

La suma ces modular en 2^{32}, si acarreo de salida.

- Cifrado, para i de 0 a 3:

$temp = R;$ $\qquad$ $R = L\ \hat{}\ F(R,\ key[i];$ $\qquad$ $L = temp$

- Descifrado, para i de 0 a 3:

$temp = L;$ $\qquad$ $L = R\ \hat{}\ F(L,\ key[i];$ $\qquad$ $R = temp$

$F4E\ par,\ kv,\ \#i$ ejecuta exactamente una iteración del lazo de cifrado y $F4D$ una del lazo de descifrado. Un bloque completo se cifra encadenando cuatro iteraciones con $i = 0,\ 1,\ 2,\ 3$ y se descifra con los índices en el orden inverso.

- Decisiones de diseño:

Desición	Alternativa descartada	Razón
Una instrucción por ronda	Una por bloque completo	El enunciado lo exige, y una cantidad de 4 rondas encadenadas tendría un camino crítico cuatro veces más largo y no permitiría enlazar bloques para ocultar la latencia
Par de registros alineado	Dos campos de registro de 4 bits	Un campo de 3 bits identifica origen y destino de ambas mitades; con 2 campos la instrucción no cabría en 16 bits
Índice de ronda como inmediato	Índice en un registro	El lazo de 4 rondas siempre se desenrolla, que es el modo natural de un VLIW. Un registro obligaría a leer un operando adicional

8. Modelo de seguridad de la bóveda:

a. Estructura:

Elemento	Descripción
Bóveda	4 llaves de 128 bits, cada una en 4 subllaves de 32 bits
$KVALID[0\ldots 3]$	Un bit por ranura; se enciende al escribirse las 4 palabras de esa llave

Registro de desafío	128 bits, solo escritura. Ninguna instrucción puede leerlo
ROT_SECRET	Secreto de raíz de confianza, 128 bits, parámetro de elaboración. En $key_vault.sv$ se implementa como el registro interno $password_reg$
$AUTH$	Bit del PSW que indica sesión abierta

b. Protocolo:

Paso	Descripción	Efecto
#1. Presentar credencial	$auth\ \#0,\ rs$ … $auth\ \#3,\ rs$	Escribe las 4 palabras del desafío
#2. Autenticar	$login$	Compara contra ROT_SECRET; fija $AUTH$ o $AUTHFAIL$
#3. Verificar	$br.auth\ etiqueta$	Bifurca según el resultado sin exponer la bóveda
#4. Instalar llave	$ksetw\ k,\ \#w,\ rs \times 4$	Requiere $AUTH = 1$
#5. Operar	$f4e\ /\ f4d$	Requiere $AUTH = 1$ y $KVALID[kv] = 1$
#6. Cerrar	$logout$	$AUTH \leftarrow 0$ y limpia el desafío

$LOGOUT$ cierra la sesión pero no borra las llaves instaladas; el borrado explícito es $KCLR$.

c. Cierre automático de sesión:

$AUTH$ se limpia sin intervención del programa cuando ocurre cualquiera de estos eventos:

Disparo	Descripción
Salida de la región segura	El PC abandona el rango $[SEC_BASE,\ SEC_LIMIT]$
Falla	Se registra cualquier cosa en el PSW

Con el primero, una rutina segura invocada con JAL queda deslogueada por hardware al retornar aunque el programador haya omitido el $LOGOUT$. Con el segundo, el programa interrumpido por una condición anómala no deja la sesión abierta.

Se evaluó un watchdog por contador de ciclos y se descarto como mecanismo principal: introduce cierres de sesión que dependen del tiempo de ejecución, lo que reduce fallas intermitentes muy difíciles de reproducir en testbench. La comparación del PC contra el rango seguro es determinista, cuesta dos comparadores y cubre el mismo escenario de programa interrumpido.

d. Aislamiento:

No existe ninguna instrucción, en ningún slot, cuya fuente sea la bóveda y cuyo destino sea un registro de propósito general o una dirección de memoria de datos. Las únicas lectoras son $F4E$ y $F4D$, y su salida ya pasó por la función de ronda, que no es invertible sin conocer la subllave. El aislamiento es estructural: en la microarquitectura no hay ruta de datos desde $key_vault.sv$ ni hacia $dmen.sv$.

Casos de prueba previstos para la Entrega 2:

- $ksetw$ sin autenticación: se anula, $CAUSE = DENIED$.
- $login$ con credencial incorrecta: $AUTHFAIL = 1$, $AUTH = 0$.
- $f4e$ sobre ranura con $KVALID = 0$: se anula, $CAUSE = NOKEY$.
- Salto fuera de la región segura con sesión abierta: $AUTH$ pasa a 0 en el mismo ciclo.

- Volcado del banco de registros y $DMEM$ tras cifrar: Ninguna palabra coincide con una subllave instalada.

Decisiones de diseño:

Desición	Alternativa descartada	Razón
Llave instalada palabra por palabra	Instrucción que instale las 4 de una vez	$KWSET$ necesita un solo puesto de lectura; la alternativa exigiría cuatro o un grupo implícito de registros que restringiría la asignación
Credencial de 128 bits	Password de 32 en un registro	Simetría con el tamaño de llave y espacio de búsqueda de 2^{128}. El costo de codificación es cero porque $AUTHW$ reutiliza la forma de $KSETW$
Desafío de solo escritura	Comparar contra un registro	Evita que la credencial quede residente en un registro de propósito general donde otro código pueda leerla
$KVALID$ por ranura	Asmir toda llave válida	Una ronda sobre una ranura nunca inicializada cifraría con ceros sin revisar, produciendo un archivo trivialmente recuperable

9. Modelo de seguridad de la bóveda:

Esta sección describe cómo se pueden combinar las instrucciones. Es la parte del documento que el grupo de CE 1108 necesita para generar código válido.

a. Regla única:

Entre el bundle que produce un valor y el bundle que lo consume debe haber al menos dos bundles de por medio. Después de todo salto se ejecuta exactamente un bundle adicional, se tome o no se tome

b. Latencias arquitectónicamente visibles:

Productor	**Latencia**
Resultado de ALU	3 bundles
Resultado de load	3 bundles
Registro base actualizado por $LW.INC$ / $SW.INC$	3 bundles
Resultado de ronda Feistel4	3 bundles
Banderas (CMP → salto)	3 bundles
Enlace $R15$ de JAL / $JALR$	3 bundles
Bit $AUTH$ ($LOGIN$ → $BR.AUTH$)	3 bundles
Delay slot de salto	1 bundle; siempre ejecutado

El valor sale del pipeline de cinco etapas de la sección 13: Un productor en el bundle n escribe el banco de registros en WB, durante el primer ciclo $n + 4$; un consumidor en el

bundle m lee en ID, durante el ciclo $m + 1$. Con escritura en el primer semiciclo y lectura en el segundo, la condición es $m + 1 \geq n + 4$, es decir $m \geq n + 3$.

La latencia es idéntica para todas las unidades. No hay tabla de casos especiales.

c. Semantica del bundle:

Regla	**Detalle**
Lecturas	Todos los slots leen el estado de registros previo al bundle. No hay reenvío intra-bundle
Escrituras	Dos slots no pueden escribir al mismo registro en el mismo bundle. El ensamblador lo rechaza; en hardware prevalece el slot de menor índice y se registra $WCONF$
Banderas	Solo S0 puede escribirlas. Una comparación en S1 registra $ILLOO$
Ejecución	Los cinco slots se ejecutan en paralelo dentro del mismo ciclo
Stalls	Ninguno por dependencias de datos. Se emite un bundle por ciclo

d. Ejemplos:

- Incorrecto, consumo antes de tiempo:

$add\ r6, r7, r8 \mid - \mid - \mid - \mid - \ ;$
$sub\ r9, r6, r7 \mid - \mid - \mid - \mid - \ ;\ //\ r6\ todavía\ no\ está\ escrito$

- Correcto:

$add\ r6, r7, r8 \mid - \mid - \mid - \mid - \ ;$
$- \quad \mid - \mid - \mid - \mid - \ ;$
$- \quad \mid - \mid - \mid - \mid - \ ;$
$- \quad \mid - \mid - \mid - \mid - \ ;$
$sub\ r9, r6, r7 \mid - \mid - \mid - \mid - \ ;$

- Incorrecto, comparación y salto demasiado juntos:

$cmpi\ r10, \#0 \mid - \mid - \mid - \mid - \ ;$
$- \quad \mid - \mid - \mid - \mid br.ne\ top\ ;\ //\ banderas\ aún\ no\ escritas$

- Incorrecto, doble escritura:

$add\ r6, r7, r8 \mid sub\ r6, r9, r10 \mid - \mid - \mid - \ ;\ //\ ambos\ escriben\ r6$

- Correcto, delay slot aprovechado:

$\mid - \mid - \mid - \quad \mid - \mid - \mid br.ne\ lazo\ ;$
$\mid - \mid - \mid sw\ r5, \#20(r13) \quad \mid - \mid - \mid - \quad ;\ //\ se\ ejecuta\ siempre$

e. Modo conservador de emisión:

Para un generador de código que no realice análisis de dependencias, la siguiente regla produce código siempre correcto:

Emitir una sola operación por bundle, en el slot que le corresponde, con los demás en NOP. Insertar dos bundles completamente vacíos entre cada par de bundles con operación, y un bundle vacío después de todo salto.

Esto expande cada instrucción frente a tres bundles. El código resultante corre a un tercio de la velocidad posible pero no requiere ningún análisis, y sirve como punto de partida verificable. La optimización consiste en ir llenando los slots y los bundles vacíos con operaciones independientes.

10. Sintaxis del ensamblador propio:

a. Estructura de línea:

Un bundle por línea, cinco campos posicionales separados por | y terminados en ;. El guión indica un slot vacío.

$[etiqueta:]$

$< S0\colon\ ALU\ -\ 0 >\ |\ < S1\colon\ ALU\ -\ 1 >\ |\ < S2\colon\ LSU >\ |\ < S3\colon\ CRP >\ |\ < S4\colon\ BRU >;\ //comentario$

La sintaxis es posicional y no por etiquetas de slot porque simplifica la emisión desde el compilador y porque cada línea fuente corresponde exactamente a una línea de 32 dígitos hexadecimales en el archivo de salida.

b. Reglas léxicas y directivas:

Elemento	**Regla**
Etiquetas	Cadena alfanumérica que inicia con Letra 0 _ seguido de letras, dígitos o _; terminan en : Van en su línea propia o al inicio del bundle (ej, ´inicio:´ o ´lazo_cifrado:´)
Registros	$r0$ a $r15$, sin distinción de mayúsculas
Pares	$p0$ a $p7$ (corresponden a los pares de registros alineados R2n:R2n+1).
Llaves	$k0$ a $k7$ (índice de llave en la bóveda de 128 bits).
Inmediatos	Precedido por # seguido de un decimal con

	signo (#-4, #16), o $0x$ y hexadecimal (0x1F)
Comentarios	Desde // hasta fin de línea
Slot vacío	Representado exclusivamente por un guión ´ – ´ . Indica que el slot no ejecuta ninguna operación en ese ciclo. El ensamblador lo traduce automáticamente al opcode binario NOP correspondiente a ese slot.
Omisión de slots	Si una línea contiene menos de 5 campos separados por ´ | ´, el ensamblador rellenará explícitamente con NOP (guión ´ - ´) los slots faltantes hacia la derecha antes del punto y coma ´ ; ´

Directiva	**Efecto**
$.text$ / $.data$	Inicia la sección de código o de datos
$.org\ 0x1000$	Fija la dirección de ensamblado
$.word$ / $.byte$	Emite una palabra de 32 bits o un byte en $DMEM$
$.space\ 64$	Reserva bytes sin inicializar
$.secure$ / $.endsecure$	Delimitan la región segura. El ensamblador verifica que el código quede dentro de $[SEC_BASE,\ SEC_LIMIT]$

c. Validaciones estáticas:

El ensamblador rechaza el programa, sin emitir binario, cuando detecta: dos slots escribiendo el mismo registro en un bundle, una comparación fuera de S0, una instrucción en un slot que no corresponde a su unidad funcional, un campo reservado distinto de cero,

un desplazamiento de salto fuera de rango, un inmediato fuera del rango del campo, o una etiqueta no definida o indefinida dos veces.

La violación de la regla de latencia se emite como advertencia y no como error, porque el generador de código puede tener información que el ensamblador no tiene, por ejemplo un registro escrito no se lee nunca.

d. Formato de salida:

Archivo de texto con un bundle por línea, 32 dígitos hexadecimales sin prefijo, apto para $readmemh. Opcionalmente un archivo de listado con la dirección, el hexadecimal y la línea fuente de cada bundle.

e. Ejemplo completo de codificación de un bundle (128 bits)

Para mostrar el mapeo directo entre la sintaxis del ensamblador y la representación final en memoria (128 bits / 32 dígitos hexadecimales), se presenta el siguiente bundle de ejemplo ejecutando las 5 unidades en paralelo:

Línea de instrucción VLIW en ensamblador:

addi r1, r1, #4 | nop | lw.inc r2, #0(r0) | f4e p0, k0, #0 | nop ;

Desglose de campos en binario por slot:

1. Slot S0 [127: 96] (ALU-0 -- 32 bits): ´addi r1, r1, #4´

- Opcode (31:27): 10001 (ADDI)
- rd (26:23): 0001 (r1)
- rs1 (22:19): 0001 (r1)
- imm19 (18:0): 0000000000000000100 (#4)
- Binario: 1000 1000 0001 0001 0000 0000 0000 0100 → 0x88110004

2. Slot S1 [95:64] (ALU-1 – 32 bits): ´nop´

- Opcode (31:27): 0000 (NOP)
- Reservado (26: 0): 00000000000000000000000000
- Binario: 0000 0000 0000 0000 0000 0000 0000 0000 → 0x00000000

3. Slot S2 [63:32] (LSU – 32 bits): ´lw.inc r2, #0(r0)´

- Opcode (31: 27): 10110 (LW.INC)
- rd (26:23): 0010 (r2)
- rbase (22:19): 0000 (r0)
- imm19 (18:0): 0000000000000000000 (#0)
- Binario: 1011 0001 0000 0000 0000 0000 0000 0000 → 0xB1000000

4. Slot S3 [31:16] (Cripto/Bóveda – 16 bits): ´ f4e p0, k0, #0´

- Opcode (15:13): 001 (F4E)
- par (12:10): 000 (p0)
- kv (9:8): 00 (k0)
- ronda (7:6): 00 (#0)
- Reservado(5:0): 00000
- Binario: 0010 0000 0000 0000 → 0x2000

5. Slot S4 [15:0] (BRU – 16 bits): ´nop´

- Opcode (15:13): 000 (NOP)
- Reservado (12:0): 0000000000000
- Binario: 0000 0000 0000 0000 0000 → 0x0000

Resultado en memoria, hexadecimal de 128 bits (32 digitos para $readmemh (Verilog)):

8811000400000000B100000020000000

11. Equivalencias para el generador de código:

Instrucciones habituales de otras arquitecturas y su construcción en nuestra arquitectura. Los bundles intermedios se omiten por claridad; aplica la regla de latencia de la sección.

Forma habitual	**Secuencia en CERBERO**

$jmp\ L$	$br.al\ L$
$jz\ L\ /\ beq\ a,\ b,\ L$	$cmp\ ra,\ rb$ en S0, luego $br.eq\ L$
$jnz\ L\ /\ bne\ a,\ b,\ L$	$cmp\ ra,\ rb$ en S0, luego $br.ne\ L$
$jl\ L\ /\ blt\ a,\ b,\ L$	$cmp\ ra,\ rb$ en S0, luego $br.lt\ L$
$jg\ L\ /\ bgt\ a,\ b,\ L$	$cmp\ ra,\ rb$ con operandos invertidos, luego $br.lt\ L$
$jle\ L$	$cmp\ ra,\ rb$ invertidos, luego $br.ge\ L$
$jge\ L$	$cmp\ ra,\ rb$ luego $br.ge\ L$
$call\ L$	$jal.al\ L$
ret	$jr.al\ L$
$push\ rx$	$subi\ r14,\ r14,\ \#4$, luego $sw\ rx,\ \#0(r14)$
$pop\ rx$	$lw\ rx,\ \#0(r14)$, luego $addi\ r14,\ r14,\ \#4$
$mov\ rx,\ \#const32$	$movi\ rx,\ \#low16$, luego $movh\ rx,\ \#high16$
$div\ ra,\ rb$	$jal.al\ _div32$, con dividendo en $R6$ y divisor en $R7$; cociente en $R6$ y residuo en $R7$

Sobre la división: CERBERO no incluye instrucción de división. Un divisor de 32 bits es multiciclo por naturaleza: implementarlo combinacional produciría el camino crítico más largo del procesador por un margen amplio, e implementarlo multiciclo obligaría a que esa instrucción tuviera una latencia distinta a todas las demás, rompiendo la única regla que constituye el contrato con el generador de código. El precedente es directo: el conjunto base de RISC-V tampoco incluye multiplicación ni división, que viven en una extensión opcional. la división se provee como subrutina $_div32$, escrita en ensamblador propio y entregada junto con el ISA. Los opcode 10011 y 11011 del slot ALU quedan reservados por si se decide incorporar en hardware.

12. Unidades funcionales:

#	Unidad	Slot	Módulo	Función
1	ALU-0	S0	alu.sv (instancia 0)	Aritmética, lógica, desplazamientos. Única que escribe banderas
2	ALU-1	S1	alu.sv (instancia 1)	Idéntica a ALU-0, sin escritura de banderas
3	LSU	S2	lsu.sv	Load/store con y sin postincremento
4	Unidad Feistel4	S3	crypto_unit.sv	Una ronda completa, combinacional
5	Controlador de Bóveda	S3	key_vault.sv	4×128 bits, autenticación, KVALID, auto-logout
6	ALU corta	S3	short_alu.sv	Dos operandos, destructiva
7	BRU	S4	bru.sv	Saltos, condicionales, enlace

Tabla de Resumen con los tipos de instrucciones y sus operaciones:

Slot	Tipo	Opcode	Mnemónicos incluidos
S0 (ALU-0)	A / I	00000 - 11111	NOP, ADD, SUB, MUL, AND, OR, XOR, SLL, SRL, SRA, ROL, ROR, MOV, NOT, CMP, TEST, NEG, ADDI, SUBI, ANDI, ORI,

			XORI, SLLI, SRLI, SRAI, ROLI, MOVI, MOVH, CMPI, TESTI
S1 (ALU-1)	A / I	00000 - 11111	Misma lista excepto CMP, TEST, CMPI, TESTI que solo son válidas en S0
S2 (LSU)	M	00000 - 11110	NOP, LB, LBU, LH, LHU, LW, SB, SH, SW, LW.INC, SW.INC
S3 (Cripto / Bóveda / ALU corta)	F/ V / AC	000 - 110	F4E, F4D, KSETW, AUTHW, VCTL (LOGIN, LOGOUT, PWSET, KCLR), y las subfunciones de la ALU corta
S4 (BRU)	C / J	000 - 101	NOP, BR (EQ, NE, LT), JAL, JR, JALR, HALT.

13. Justificaciones de Diseño:

#	Desición	Alternativa descartada	Razón
1	Bundle de 128 b con 5 slots	4 slots uniformes	El lazo de cifrado necesita cinco operaciones por ciclo
2	Anchos de slot desiguales	32 b uniformes	BRU y unidad criptográfica usan índices

			pequeños; ataca la baja densidad de código típica de VLIW y ahorra memoria de instrucciones
3	Espacio de opcodes por slot	Mapa global de opcodes	La posición del slot ya identifica la unidad funcional
4	Opcode contiguo de 5 b	Campos de función separados estilo funct3/funct7	No compartimos formato entre slots, así que no pagamos el costo de alinear inmediatos; el decodificador es de un nivel y los inmediatos quedan contiguos
5	16 registros sin cero cableado	8 registros, o R0 = 0	Cinco slots por ciclo ahogan un banco de 8; con R0 = 0 el par P0 quedaría degradado, y sobra espacio de opcode para definir MOV, NOT y NEG como instrucciones reales
6	Sin punto flotante ni división	Unidades dedicadas	El dominio es cifrado por bloques y datos enteros; ambas romperían la latencia uniforme
7	ALU corta en S3	S3 exclusivo para cifrado	Sin ella, todo programa de propósito general desperdicia un quinto de cada bundle
8	Arquitectura Harvard	Memoria unificada	El ancho de bundle no coincide con el de los datos; simplifica la herramienta de carga de archivos, se evita un cuello de botella al tener datos e instrucciones en la misma memoria
9	Postincremento en la LSU	Solo base+desplazamiento	Acceso y avance de puntero en un ciclo, liberando las dos ALU. El costo es un puerto de escritura adicional
10	Saltos por banderas	Compare-and-branch	Permite un slot BRU de 16 bits y resolución temprana con un solo delay slot
11	Solo S0 escribe banderas	Ambas ALU	Evita el conflicto de escritura sobre el PSW
12	Un delay slot explícito	Vaciado de pipeline	Sin penalización oculta, coherente con la calendarización estática

13	Latencia uniforme de 3 bundles	Latencias por unidad	El contrato con CE 1108 cabe en una línea
14	Auto-logout por región segura	Watchdog por contador de ciclos	Un contador produce cierres intermitentes difíciles de depurar en testbench

.